

AERO40003 Computing and Numerical Methods 1

Numerical Methods Coursework 2026

Version 01

Dr. Roderick Lubbock

Completing this live file

You should submit only this self-contained completed livefile. No other scripts or other files will be marked.

When completing this livefile:

- Please make sure all of your code is formatted as code using the "code" button on the Live Editor ribbon;
- Please make sure anything that is not code or code comments is formatted as text;
- Marks for each section and sub-section are shown in bold square brackets;
- Ensure that the entire livefile runs without errors before submitting! Sections of the file which do not run will not be marked.
- Warnings when running the code will not be penalised, however errors will be penalised.

Code

EVERY LINE OF CODE (with the exception of plot commands and loop end statements) must have a comment clearly explaining what it does. Comments should be brief but clear. They do not need to be complete sentences, in many cases they can be just a few words.

All self-contained functions must:

- be entered in the **Functions** section at the **bottom** of this livefile;
- be named correctly according to the guidance below;
- have a comment on each line of code explaining its function and how it relates to the equations in the Class Slides or other theory;
- NOT call built-in MATLAB functions from outside the standard MATLAB toolbox e.g. C:\Program Files\MATLAB\R2024b\toolbox\matlab (but you may use these to check your functions).

To check the toolbox of a MATLAB function use the which command. The short name of the toolbox is shown after "toolbox" For example:

```
which spline
```

returns e.g. C:\Program Files\MATLAB\R2024b\toolbox\matlab\polyfun\spline.m which means it is in the standard MATLAB toolbox, whereas

```
which fit
```

returns e.g. C:\Program Files\MATLAB\R2024b\toolbox\curvefit\curvefit\fit.m which means it is in the curvefit toolbox, and not the standard MATLAB toolbox.

Plots

You should ensure any plots you are asked to produce have:

- axis labels;
- x and y grid;
- all lines/markers clearly differentiable through use of colour or line/marker type;
- legend (where more than one dataset is plotted), placed so as to minimise overlap with data lines/markers;
- any spaced discrete point data (such as that in the table below) plotted as points (not lines);
- when plotting overlapping functions use solid, dashed, chain-dashed and dotted linetypes to maximise visibility of all lines.

Note that any plots that are not formatted as above (and exemplified in the Computer Labs Code livefiles) **will be penalised.**

Questions

- All **questions** (Q1.1, Q1.2 etc.) should be answered using written answers formatted as text, NOT code, in the spaces provided in each section. Any code submitted as answers to questions will be ignored. You may of course refer to further plots or code in the **Code** section to aid your explanations.

Submission

- **You should submit only this self-contained completed livefile. No other scripts or other files will be marked.**
- You should submit your livefile on Blackboard by the deadline stated on Blackboard.
- Please rename the file in the format **NM1_CW_XXXXXX.mlx** where **XXXXXX** is your CID number.
- As usual, please ensure you receive a confirmation email following submission. Only confirmed submitted files will be marked (partially complete Blackboard submissions will be ignored).
- **Note that submissions may be checked for plagiarism using multiple plagiarism detection softwares for similarity to other submissions.**
- **If you are trying to decide whether to plagiarise content or submit incomplete content, the choice should always be the latter, because a low grade is better than no grade, and also avoids the stress (for you) and extra work (for departmental staff) involved in plagiarism investigations.**

Plagiarism

Discussion with fellow students is encouraged, however all material that is submitted for assessment must be the result of your own individual intellectual effort and must accurately and substantively reflect your understanding of the subject matter at the time of submission. **Note that submissions may be checked for similarity to other submissions in this and previous years using multiple plagiarism detection softwares.**

You may be wondering how you should consult/engage with fellow students or large language models without committing plagiarism?

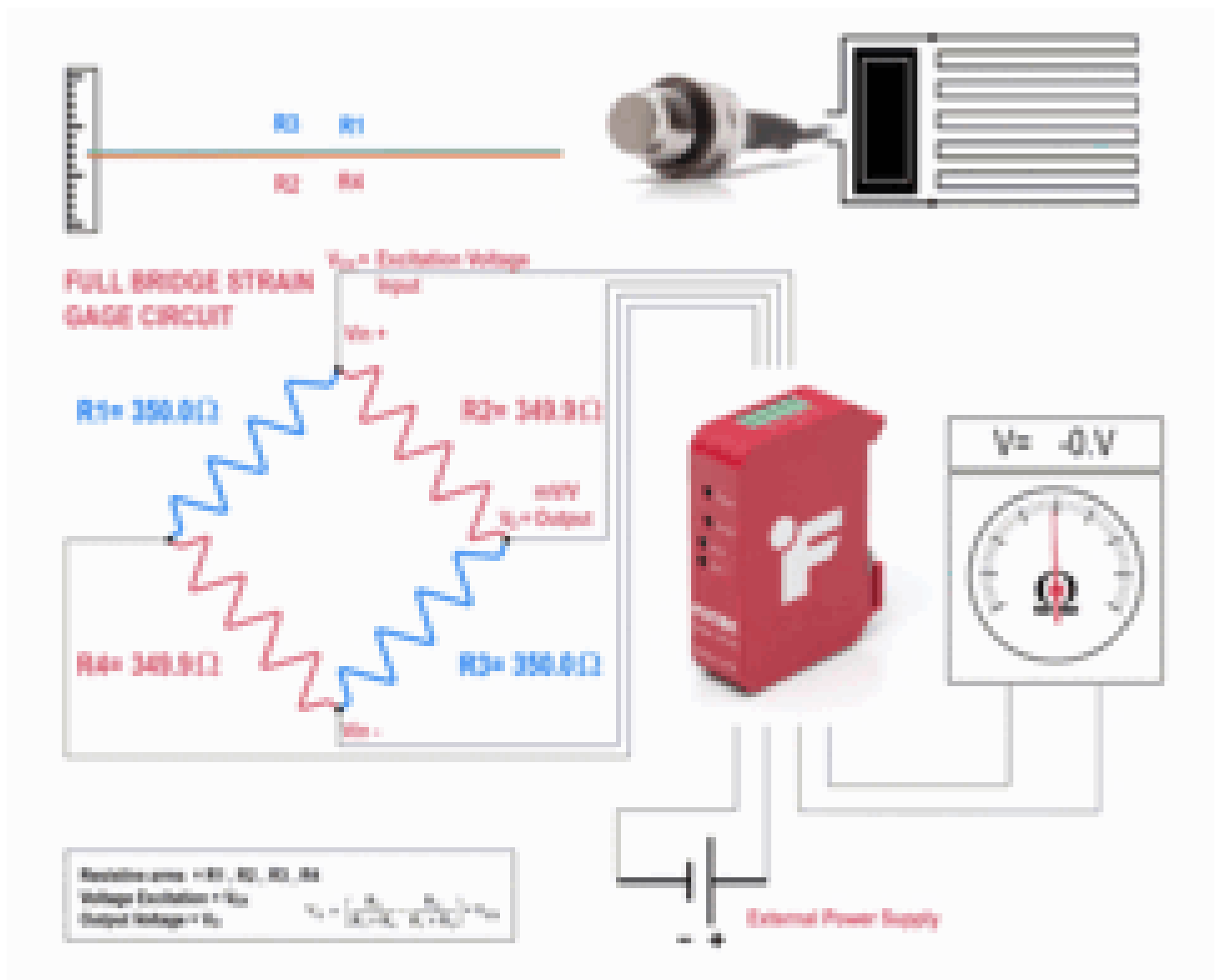
- Discussing problem solving methods and approaches with fellow students / LLMs, is not considered plagiarism.
- Asking fellow students / LLMs to show you how to complete specific problems or to write code for you, and then submitting copies of this code as your own work **is considered plagiarism**.

As a general academic rule: **anything that you publish or submit that is not generated/authored by you should be referenced**, with the reference giving details of the author(s). **If in doubt: reference.** For example, although it is not recommended as it doesn't help you learn, if you copy part of a solution from a fellow student/LLM in order to complete further parts of a problem, and you reference it, then this would not be considered plagiarism (although you won't get any marks for it).

If you are trying to decide whether to plagiarise content or submit incomplete content, the choice should always be the latter, because a low grade is better than no grade, and also avoids the stress (for you) and extra work (for departmental staff) involved in plagiarism investigations.

Introduction

The aim of this coursework is to use various numerical methods to understand how circular things vibrate. One such thing that does this, and that as an aeronautical engineer you will come across frequently, (i.e. is installed on every aircraft that you have ever flown on), is a pressure transducer. How does a pressure transducer work? Essentially it is a circular membrane that deforms in response to applied fluid pressure. The deformation is measured using strain gauges bonded to the diaphragm. As the diaphragm deforms, the arms of the strain gauge are strained and this changes their resistance, which changes the output voltage of the gauge (which is usually arranged as a [Wheatstone bridge](#) to maximise *sensitivity*). This is illustrated in the animated .gif image below, where you can see the flexing diaphragm half-x-section (upper left) vs the strain gauge strain (upper right) vs the wheatstone bridge resistances (lower left) vs the voltage reading (lower right). [This 2-minute youtube video made by Omega](#), one of the largest global pressure sensor manufacturers, also gives a nice explanation of how a pressure transducer works.



Animation of pressure transducer function (at max gif resolution for a ML livefile). See <https://www.futek.com/pressure-transducer> for a higher resolution version.

Pressure isn't always steady, very often it fluctuates, for example in the turbulent airstream in the wake of a formula one car. Transducers used to characterise this turbulence (e.g. on the nose of the car behind) need to be able to follow the rapid fluctuations. This is a problem, because if a transducer is subjected to pressure fluctuations around its resonant frequency, the output may be many times the actual pressure fluctuation amplitude. In other words, a transducer cannot measure fluctuations in the region of its natural frequencies. To be able to design a transducer with a useful *measurement bandwidth* (frequency measurement range before resonance), we need to be able to predict the resonant frequencies.

A pressure transducer diaphragm vibrates in the same way as an even simpler device: a musical drum. The displacement of the circular drum skin is a function of time, radius and angle (in polar coordinates obviously), and is described by the following partial second order differential equation:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \left(\frac{\partial^2 u}{\partial r^2} + \frac{1}{r} \frac{\partial u}{\partial r} + \frac{1}{r^2} \frac{\partial^2 u}{\partial \theta^2} \right) \text{ for } 0 \leq r < a, 0 \leq \theta \leq 2\pi$$

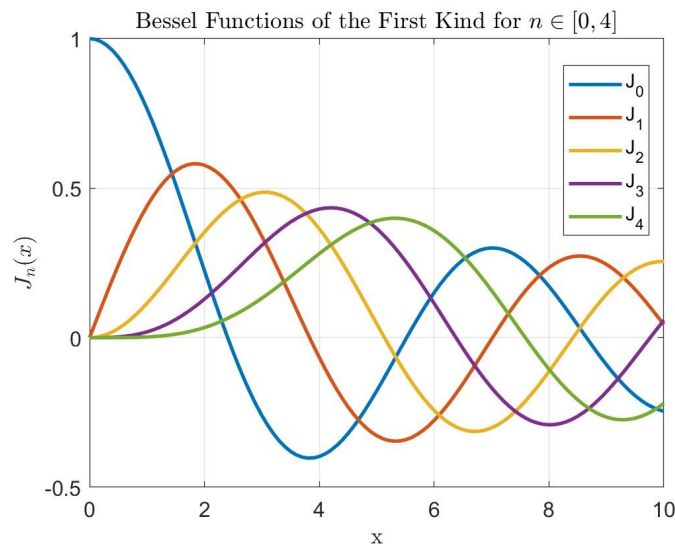
Solutions to this equation are given by the following expression:

$$u_{mn}(r, \theta, t) = (A \cos c\lambda_{mnt} + B \sin c\lambda_{mnt}) J_m(\lambda_{mn}r) (C \cos m\theta + D \sin m\theta) \text{ for } m = 0, 1, 2, 3... \text{ and } n = 1, 2, 3...$$

This is obviously one of the unpleasant equations we were talking about in lectures, but you should understand the various components of the solution, which are a bit like a decaying sinusoidal solution for a damped harmonic oscillator in that we have a temporal part (the cos and sine terms in the first bracket are a function of time only) and a spatial part (everything else), that gives the displacement as a function of radius and angle. But you are probably wondering what the J_n thing is...

Bessel functions describe electric fields, mechanical vibrations, heat conduction, optical diffraction and other phenomena involving cylindrical or spherical symmetry including vibrating circular membranes, such as pressure transducer sensing elements and drum skins, as well as temperature distributions in heated cylindrical objects, such as combustor temperature probes and hot dogs in boiling water. Everything you could possibly want to know about Bessel functions can be found at the [DLMF: NIST Digital Library of Mathematical Functions](#), which is based on a [famous mathematical handbook](#) known as "Abramowitz and Stegun", [available as a pdf here](#).

There are various different *kinds* of Bessel functions, including the *first kind* $J_n(x)$, the *second kind* $Y_n(x)$ and the *third kind* $H_n(x)$. For each kind there are also different orders, e.g. integers $n = 0, 1, 2, 3, \dots$. The Bessel functions of the first kind for the first five integer orders ($n = 0$ to 4) are plotted below. They look a bit like decaying sinusoids, except for some subtle differences as we shall see.

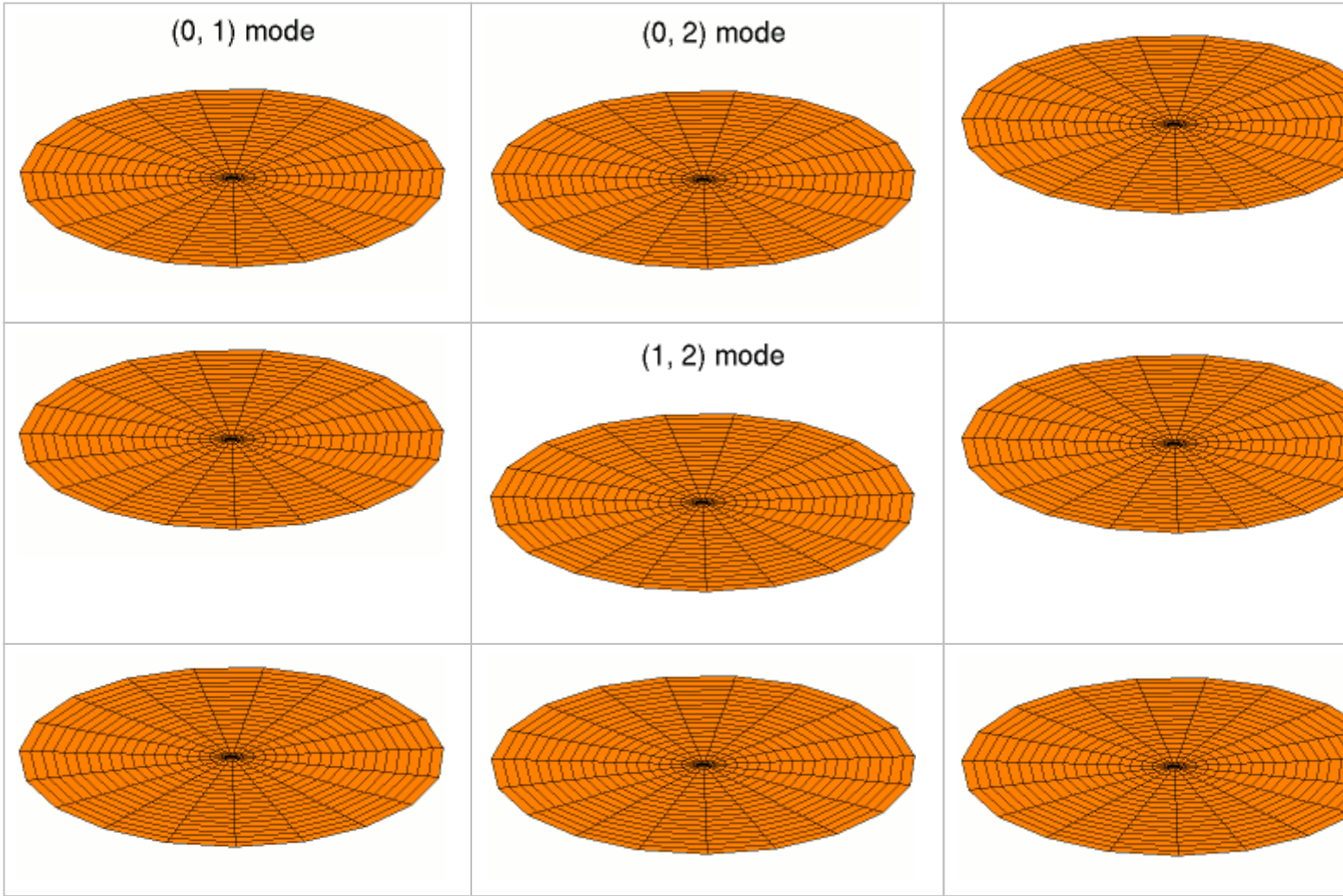


To see how this plot was made run the following code:

```
openExample('matlab/PlotBesselFunctionsOfFirstKindExample')
```

Various different vibration modes of a cylindrical drum skin or pressure transducer diaphragm are illustrated in the animated gifs below (source: https://en.wikipedia.org/wiki/Vibrations_of_a_circular_membrane). Just such a diaphragm is shown in the following experiment on youtube: [Vibrating circular membrane](#). The fundamental

(0,1) mode is shown in the top left, which corresponds to the first zero of $J_0(x)$, whilst the (0,2) mode shown in the top middle corresponds to the second zero of $J_0(x)$, and the (0,3) mode in the top right to the third zero. The modes shown on the second row of images correspond in a similar way to the zeros of $J_1(x)$, whilst the third row correspond to zeros of $J_2(x)$. You should be able to convince yourself of the above by looking at the locations of the nodes and anti-nodes in the animated images (and video) and seeing how these correspond to the Bessel function plot above. Remember that $J(\lambda_{mn}r)$ only describes the displacement as a function of radius, so consider what the x-section of the oscillating diaphragms looks like for each mode.



Looking at the solution given above it is apparent that the oscillation angular frequencies are given by the t -multiplier in the sin and cos terms of the temporal part, i.e. $\omega_{mn} = c\lambda_{mn}$. Since the edge of the diaphragm must be a node, this means that $\lambda_{mn}a$ must be a zero of the m th order Bessel function $J_m(x)$, i.e. α_{mn} , where n is the root counter i.e. $n = 0, 1, 2, \dots$. This means we can write the frequencies of the modes as $\omega_{mn} = \frac{\alpha_{mn}c}{a}$, where

$c = \sqrt{\frac{N}{\rho h}}$. In order to find the resonant frequencies of a circular diaphragm we therefore need to be able to

find the roots of various Bessel functions. Furthermore, it is useful to develop functions that describe the radial displacement of circular diaphragms. For example, we may also wish to know the (maximum) displacement at the anti-nodes vs. frequency etc.

Part 1: Generating Bessel functions via interpolation [7 marks]

The problem with Bessel functions is that, unlike exponentials and sines and cosines, there isn't a Bessel function button on your calculator, and they are not very amenable to calculation by hand. Instead, they are often compiled in standard mathematical tables. One such table is **table 9.1** on **pages 390-396** of Abramowitz and Stegun that gives Bessel functions of the first and second kind, $J_n(x)$ and $Y_n(x)$ respectively, for orders $n = 0, 1, 2$.

In this section you should interpolate the tabular data using 0.05 spacing across the range, i.e. $0 \leq x \leq 5$, in other words 101 points across this range. You will need to read off these values from the table in Abramowitz and Stegun on page 390 (5 d.p. should be sufficient) and enter them into a table in MATLAB with column headings **x** and **J**. You may be able to find a clever way to do this using online PDF tools, or importing the OCR-converted pdf page into Microsoft Excel and then using the "paste excel data" context menu option in the MATLAB variables window.

Investigate the different methods of interpolation covered in the course to evaluate the **zeroth order Bessel Function of the first kind**, $J_0(x)$ as follows:

- **M1.1:** Write a self-contained MATLAB function **called IntPoly** that fits a polynomial of the maximum possible order to the tabular data using the first approach in Class 4 involving inversion of a matrix of x -values. **[1]**
- **M1.2:** Write a self-contained MATLAB function **called LagrangePoly** to compute the interpolating Lagrange polynomial that passes through all interpolant data points. **[1]**
- **M1.3:** Using the function **LagrangePoly** interpolate data using Chebyshev spacing for 21 points across the data range. You will need to calculate the spacing rounded to the nearest 0.1 so that you can select the nearest data point from table 9.1 **[1]**
- **M1.4:** Use the built-in MATLAB function `polyfit` to fit a polynomial of the maximum possible order to the tabular data. **[0.5]**
- **M1.5:** Write a self-contained MATLAB function **called IntSpline** that performs cubic spline interpolation with natural (free-runout) end conditions. **[1]**
- **M1.6:** Use the built-in MATLAB function `spline` to fit cubic splines to the tabular data. **[0.5]**

All functions should:

- take as input the tabular data as well as a vector of points at which the polynomial is to be evaluated.
- output the evaluation of the polynomial at the input vector points.

Evaluate the performance of the interpolated functions (plotted as a continuous curve) alongside the tabular data (plotted as discrete points) across the range of the interpolated data (i.e. $0 \leq x \leq 5$) as follows:

- **E1.1:** Plot M1.1 to M1.4 together on a single plot alongside the interpolated tabular data. **[0.5]**

- **E1.2:** Plot M1.5 and M1.6 together on a single plot alongside the interpolated tabular data. **[0.5]**

Questions

- **Q1.1:** Methods M1.1., M1.2 and M1.4 should generate equivalent (i.e. the same) polynomials according to Class 4. Investigate whether this is the case and explain any differences. You may wish to plot the absolute differences across the interval. **[0.5]**
- **Q1.2:** Compare the cubic spline methods M1.5 with M1.6. Are the outputs the same? Explain any differences (You may wish to plot the absolute differences across the interval.)**[0.5]**

Hints

Method M1.2: You should use a for loop to step through the product in the basis functions and another for loop to sum the scaled basis functions to get the Lagrange polynomial.

Part 1: Your Solution

Code (inc. Plots)

N.B.: Self-contained functions should be entered in the **Functions** section at the **bottom** of this livefile.

```
x = [0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1 1.1 1.2 1.3 1.4 1.5 1.6 1.7
1.8 1.9 2 2.1 2.2 2.3 2.4 2.5 2.6 2.7 2.8 2.9 3 3.1 3.2 3.3 3.4 3.5 3.6 3.7
3.8 3.9 4 4.1 4.2 4.3 4.4 4.5 4.6 4.7 4.8 4.9 5]; %importing x values from
table

J0 = [1 0.9975 0.99002 0.97763 0.96039 0.93846 0.912 0.8812 0.84628 0.80752
0.76519 0.71962 0.67113 0.62008 0.56685 0.51182 0.4554 0.39798 0.33998
0.28181 0.22389 0.1666 0.11036 0.05553 0.0025 -0.04838 -0.0968 -0.14244
-0.18503 -0.22431 -0.26005 -0.29206 -0.32018 -0.34429 -0.36429 -0.38012
-0.39176 -0.39923 -0.40255 -0.40182 -0.39714 -0.38866 -0.37655 -0.36101
-0.34225 -0.32054 -0.29613 -0.26933 -0.24042 -0.20973 -0.17759]; %importing
0th order Bessel values

J1 = [0 0.04994 0.0995 0.14832 0.19602 0.24226 0.2867 0.32899 0.36884
0.40594 0.44005 0.4709 0.49828 0.52202 0.54194 0.55793 0.56989 0.57776
0.58151 0.58115 0.57672 0.56829 0.55596 0.53987 0.52018 0.49709 0.47081
0.4416 0.4097 0.37542 0.33905 0.30092 0.26134 0.22066 0.17922 0.13737
0.09546 0.05383 0.01282 -0.02724 -0.06604 -0.10327 -0.13864 -0.17189
-0.20277 -0.23106 -0.25655 -0.27908 -0.29849 -0.31469 -0.32757]; %1st order
Bessel

J2 = [0 0.00125 0.00498 0.01116 0.01973 0.0306 0.04366 0.05878 0.07581
0.09458 0.1149 0.13656 0.15934 0.18302 0.20735 0.23208 0.25696 0.28173
0.30614 0.32992 0.35283 0.37462 0.39505 0.41391 0.43098 0.44605 0.45897
0.46956 0.47768 0.48322 0.48609 0.4862 0.48352 0.47803 0.46972 0.45862
0.4448 0.42832 0.4093 0.38785 0.36412 0.33829 0.31053 0.28105 0.25008
0.21784 0.18459 0.15057 0.11605 0.08129 0.04656]; %2nd order Bessel

%-----M1.1 -----v
```



```

x_matrix = x; %function input 1
bessel_matrix_data = J0; %function input 2
x_for_matrix_plot = linspace(0,5,101); %creating a plotting axis

matrix_approx = Intpoly(x_matrix,bessel_matrix_data); %calling function,
assigning to variable

```

Warning: Matrix is close to singular or badly scaled. Results may be inaccurate. RCOND = 9.777698e-45.

```

%-----M1.2 -----v
scatter(x,J0,'x') %Bassel Data from table
hold on %allowing further plots
x_L_input = x; %assigns function input to variable
x_for_Lagrange = linspace(0,5,101);
L = Lagrangepoly(x_L_input,J0); %assigns function output to variable

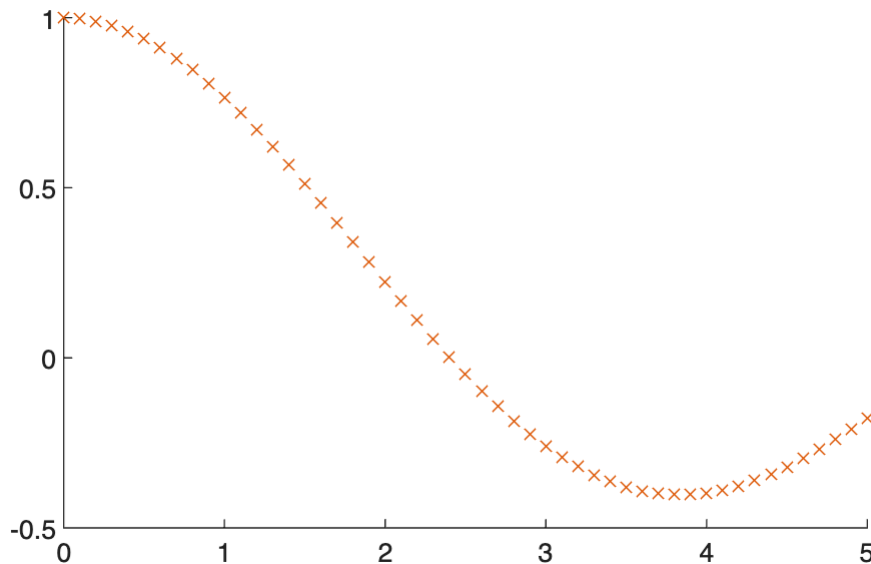
%M1.3 -----v
x_chebyshev = zeros(1,21); %initializing an empty 0 vector
x_chebyshev_alt = zeros(1,21);%initializing an empty 0 vector
for cheb = 1:21 %iterating over all 21 x points

    x_chebyshev(cheb) = -cos(pi*((2*cheb-1)/(2*21)));%chebyshev normal
formula

    x_chebyshev_alt(cheb) = round((5/2)*(x_chebyshev(cheb) +
1),1);%accounting for different limits
end

scatter(x,J0,'x'); %plotting Bessel data from table
hold on %allows for further plotting on figure

```



```
x_L_input = x_chebyshev_alt; %Assigns function input to the derived
chebyshev nodes

%Finding nearest number in table
J0_chebyshev = zeros(size(x_chebyshev_alt)); %Creating an empty matrix of
0s so there isn't constant size changes

for i = 1:length(x_chebyshev_alt) %looping over the created chebyshev nodes
    found_index = find(x == x_chebyshev_alt(i)); %identifying where in x
the chebyshev node is (as an index)
    J0_chebyshev(i) = J0(found_index); %forming a vector of points
evaluated at each chebyshev node
end

L_cheby = Lagrangepoly(x_L_input,J0_chebyshev); %assignment of lagrange
output vector to variable L

%-----M1.4 -----v
p = polyfit(x,J0,50); %assigning co-efficients from polyfir to variable p
```

Warning: Polynomial is badly conditioned. Add points with distinct X values, reduce the degree of the polynomial, or try centering and scaling as described in HELP POLYFIT.

```
x_101 = linspace(0,5,101); %forming a series of 101 x points to plot
polynomial_fit = polyval(p,x_101); %Evaluating the polynomial with
coefficients form p, to form a vector for plotting

%-----M1.5 -----v
x_spline_input = linspace(0,5,101); %forming a series of 101 x points to
plot
bessel_data = J0; %Assigning function input the J0 data from table
```

```
y_spline = IntSpline(x_spline_input,bessel_data); %Assigning y_spline the
vector which will be used for plotting later
```

```
%-----M1.6 -----v
```

```
xx = 0:0.05:5; %Assignment of a vector for spline plotting (seen later)
s = spline(x,J0,xx); %Assigning spline vector to s, which will be used for
plotting
```

```
%-----E1.1 -----v
```

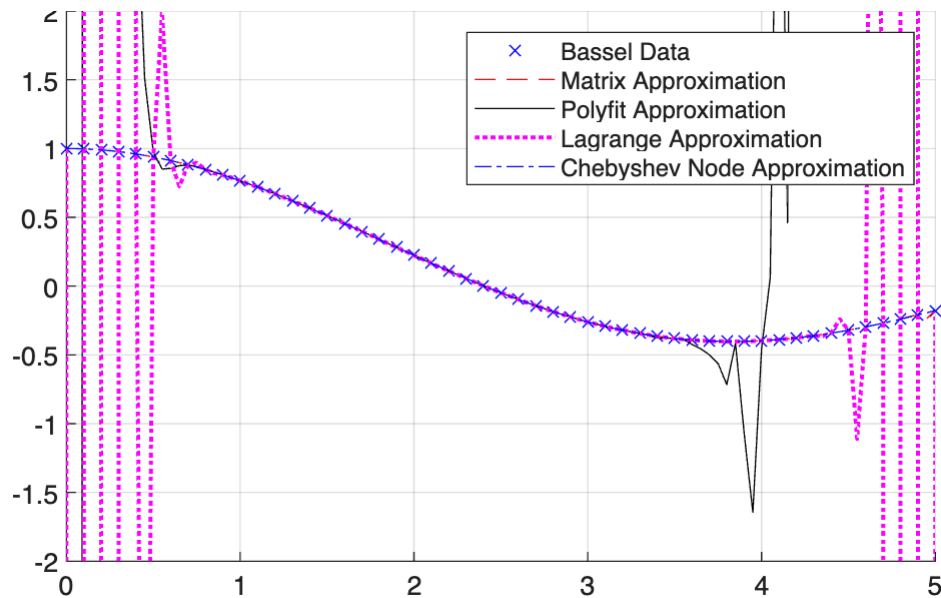
```
figure %creating new plotting space
scatter(x,J0,'x','b'); %Plotting original Bessel function data from table
hold on %allowing for further plots
y_matrix = Intpoly(x,J0); %Assignment of M1.1 vector output from function
```

Warning: Matrix is close to singular or badly scaled. Results may be inaccurate. RCOND = 9.777698e-45.

```
x_for_Matrix = linspace(0,5,101); %forming 101 x points to plot
plot(x_for_Matrix,y_matrix,'r','LineStyle','--') %Plotting the matrix
inverted polynomial
hold on %allowing for further plots
plot(x_101,polynomial_fit,'k') %Plotting polyfit/polyval polynomial
hold on
plot(x_for_Lagrange,L,'m','LineStyle',':','LineWidth',1.5)
hold on
plot(x_for_Lagrange,L_cheby,'b','LineStyle','-.')

ylim([-2,2]) %Setting limits such that the high fluctuations in polynomial
values do not affect readability
grid on %Applying grid backdrop

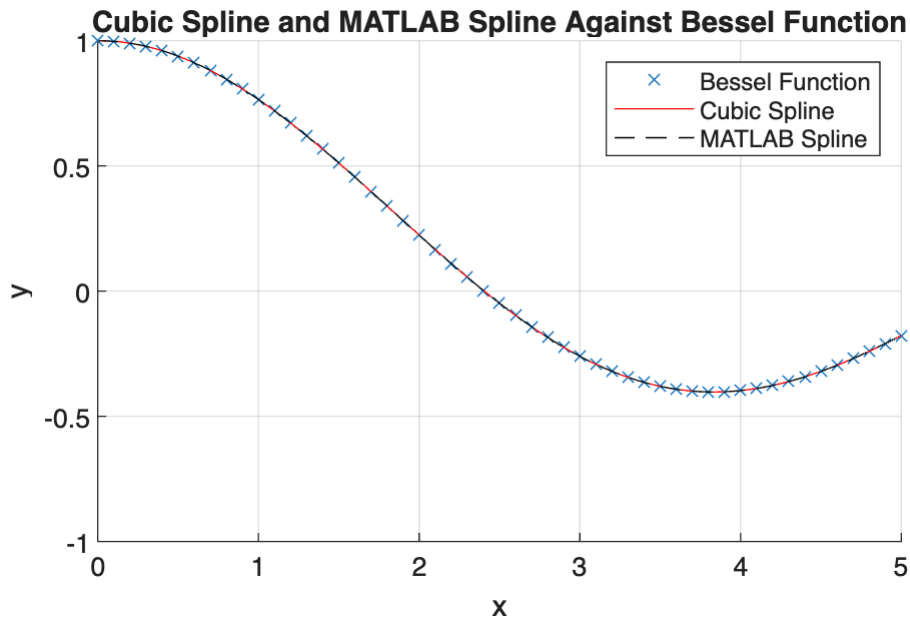
legend('Bassel Data','Matrix Approximation','Polyfit
Approximation','Lagrange Approximation','Chebyshev Node Approximation')
%Provides legend
title("Comparison of Polyfit and Matrix Approximations of Bessel Function")
hold off %Preventing further plots
```



```
%-----E1.2 -----v
```

```
figure %creating new plotting space
scatter(x,J0,'x'); %Plotting original Bessel function data from table
hold on %Allowing for further plots
plot(x_spline_input,y_spline,'r') %Plotting IntSpline function output vector
hold on %Allowing for further plots
plot(xx,s,'k','LineStyle','--') %Plotting matlab spline function vector
hold off %Preventing further plots

ylim([-1,1]) %Setting limits such that the high fluctuations in polynomial
values do not affect readability
legend("Bessel Function","Cubic Spline","MATLAB Spline") %Provides legend
grid on %Applying grid backdrop
xlabel('x') %Labelling x axis
ylabel('y') %Labelling y axis
title("Cubic Spline and MATLAB Spline Against Bessel Function") %Give a
title to plot
```



```
%-----Code to support Q1.1-----v
```

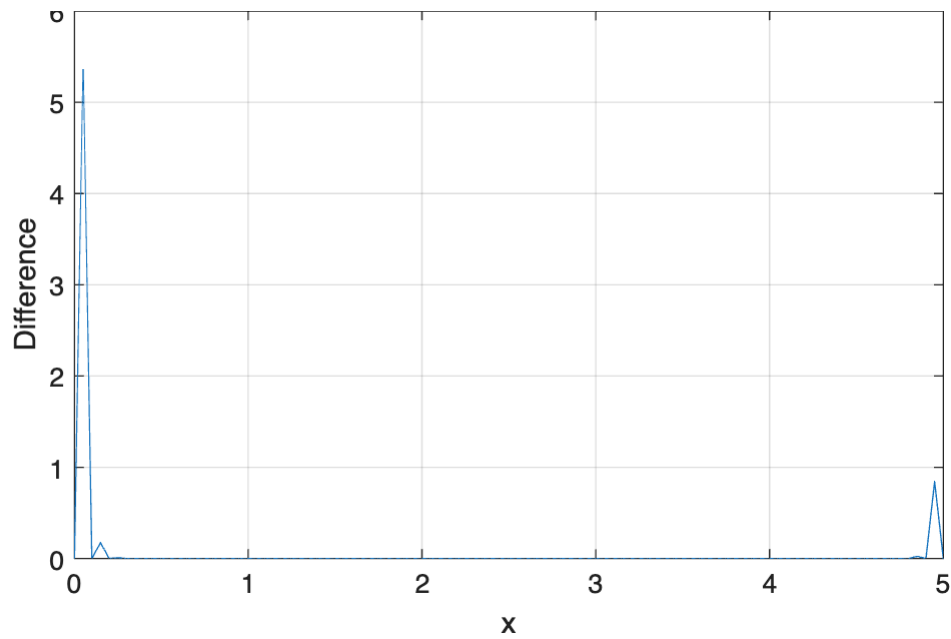
```
matrix_diff = abs(matrix_approx - L)
```

```
matrix_diff = 1×101
105 ×
    0    5.3570    0.0000    0.1739    0.0000    0.0096    0.0000    0.0008 ...
```

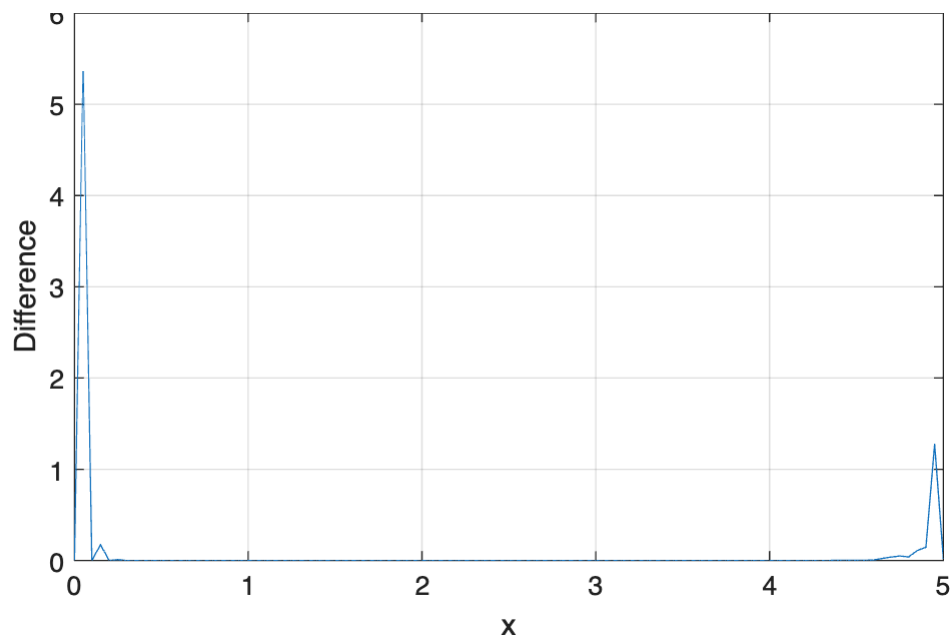
```
polyfit_diff = abs(polyomial_fit - L)
```

```
polyfit_diff = 1×101
105 ×
    0.0041    5.3561    0.0002    0.1735    0.0004    0.0099    0.0001    0.0007 ...
```

```
figure
plot(x_for_Lagrange,matrix_diff)
xlabel('x')
ylabel('Difference')
title("Absolute Difference in Lagrange and matrix inversion function")
grid on
```

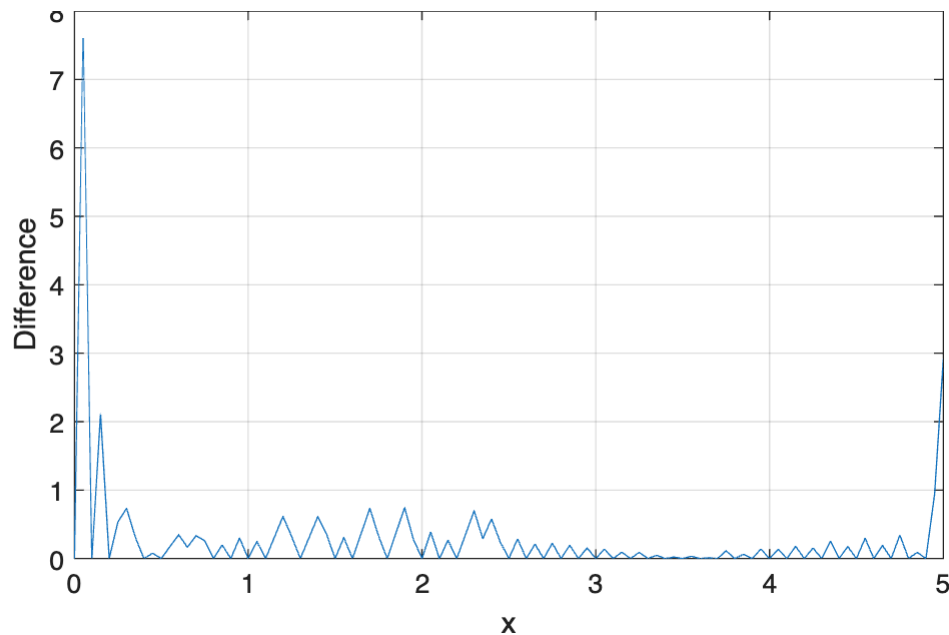


```
figure
plot(x_for_Lagrange,polyfit_diff)
xlabel('x')
ylabel('Difference')
title("Absolute Difference in Lagrange and polyfit function")
grid on
```



```
%-----Code to support Q1.2-----v
spline_diff = zeros(1,101); %preventing constant size change
for i = 1:101 %looping over all x points
    spline_diff(i) = abs(s(i)-y_spline(i)); %Finding the positive difference
end
x_spline_comapre = linspace(0,5,101);
```

```
figure
plot(x_spline_compre,spline_diff)
xlabel('x')
ylabel('Difference')
title("Absolute Difference in IntSpline and MATLAB spline function")
grid on
```



Answers to Questions

Q1.1

We see that the exact polynomial is not found across all 3 methods. Considering the Lagrangian polynomial method as our base by which we compare our other functions, we find that the matrix approximation matches the Lagrange polynomial well, however deviates towards the end. The same result is found for the poly fit function, however the difference is significantly greater. This occurs towards the end of the interpolation due to Runge's Phenomenon, i.e. the oscillation of high degree polynomials. Because we see the absolute difference, this appears as a positive spike in our difference graph.

Q1.2

The built in MATLAB spline function and the programmed cubic spline function IntSpline match, with differences of the order of 10^{-4} , as seen in the figure plotted by the code above. Most significant fluctuations arise towards the end points of the Bessel function. This may be due to the fact that the IntSpline uses a natural spline at its endpoints, i.e. the second derivative = 0 at the ends, whereas the MATLAB spline function enforces the third derivative = 0. As for intermediate errors, these may be due to precision of matrix calculations in the IntSpline function. The aforementioned boundary differences will also likely propagate to affect the function between 0 and 5 as well, further leading to differences at these points.

Part 2: Generating Bessel functions by numerical integration [7 marks]

For integer values of n , instead of tables, the Bessel function of the first kind may also be found from the integral form using *Bessel's integrals* as:

$$J_n(x) = \frac{1}{\pi} \int_0^\pi \cos(n\tau - x \sin \tau) d\tau$$

Investigate the different methods of numerical integration covered in the course to evaluate Bessel's integral for the **zeroth order Bessel Function of the first kind**, $J_0(x)$ as follows:

- **M2.1:** Write a self-contained MATLAB function **called trapzJ** that uses the Trapezium rule. [1]
- **M2.2:** Write a self-contained MATLAB function **called simpsonJ** that uses Simpson's 1/3 rule. [1]
- **M2.3:** Write a self-contained MATLAB function **called gaussJ** that uses Gauss-Legendre quadrature. [1]
- **M2.4:** Write a simple loop to evaluate the integral for the vector x using the built-in MATLAB function `integral`. To save you inputting more tabulated Bessel function data for comparison, you can take this method to be accurate to machine precision. [1]

All functions should:

- take as input: the integer order of the Bessel function (i.e. n), the number of points (nodes) to be used in the integration, N , and an input vector x that is evenly spaced by 0.1 on the interval $0 \leq x \leq 6$.
- output $J_n(x)$.

You should evaluate the performance of your self-contained functions as follows:

- **E2.1:** Plot the **absolute error** vs x on a logarithmic ordinate axis across the interval $0 \leq x \leq 6$ for methods M2.1, M2.2 and M2.3 compared to M2.4 for the following numbers of nodes: $N = [5, 11, 17, 19, 35, 37, 51]$. You should use a loop to plot all methods on one figure for each node number, starting with $N=5$. [1]

Questions

- **Q2.1:** What is the minimum number of nodes, and hence computation time, required to achieve double-precision accuracy for each integration method M2.1, M2.2 and M2.3? [1]
- **Q2.2:** You should find the absolute error plots for **trapzJ** for $N > 5$ appear to be missing data. Explain why this is. [1]

Hints

Method M2.1: Use the built-in MATLAB function `trapz`.

Method M2.2: Use an anonymous function (as in the Computer Lab Codes), that takes x and τ as the inputs, in order to define the integrand.

Method M2.3: You will need to evaluate the Legendre polynomials to higher orders than in the Class slides.

One way to approach this is to use the MATLAB functions `conv` and `polyder` to evaluate the definition of $L_N(x)$ in Class 3:

$$L_N(x) = \frac{1}{2^N N!} \frac{d^N}{dx^N} (x^2 - 1)^N$$

The $(x^2 - 1)^N$ term can be evaluated using the built-in MATLAB function `conv`, which multiplies two polynomials and returns the coefficients of the resulting polynomial (a process known as convolution):

```
pn = [1 0 -1];           % Polynomial coefficients for polynomial x^2-1
res = 1;                  % Initialise the result
N = 3;                    % Power to which you want to calculate the input polynomial
for k1 = 1:N
    res = conv(res, pn); % conv . The first iteration will simply multiply 1 and [1 0 1] to
    obtain [1 0 1].
end
% res will now be the coefficients of (x^2-1)^N expanded as a polynomial.
```

The coefficients of the derivative of a polynomial can be found using built-in MATLAB function `polyder`. You will need to code a loop similar to the one for `conv` to calculate the N th derivative. You can also use `polyder` and `polyval` to evaluate the weights.

To obtain the roots of the polynomial based on the coefficient simply use the built-in MATLAB function `roots`. To evaluate a polynomial for a given vector of input values simply use the built-in MATLAB function `polyval`.

Another way to generate the Legendre polynomials is using Bonnet's recursion formula:

$$(N + 1)L_{N+1}(x) = (2N + 1)xL_N(x) - NL_{N-1}(x)$$

Starting with $L_0(x) = 1$ and $L_1(x) = x$ this allows all subsequent polynomials to be computed. For example, $L_2(x)$ is found from:

$$L_2(x) = \frac{1}{2}(3xL_1(x) - L_0(x)) = \frac{1}{2}(3x^2 - 1).$$

Based on this it is possible to code a recurrence relation to calculate the coefficients of any Legendre polynomial. To illustrate this consider the coefficients of the first few polynomials:

$$L_0(x) = 1 :$$

$$L0_coeffs = 1$$

$$L_1(x) = x :$$

$$L1_coeffs = [1 \ 0] = [L0_coeffs \ 0]/1$$

$$L_2(x) = \frac{1}{2}(3x^2 - 1) :$$

$$L2_coeffs = 1/2*[3 \ 0 \ -1] = (\ 3*[L1_coeffs \ 0] - [0 \ 0 \ L0coeffs])/2$$

You should be able to spot a pattern forming that corresponds to Bonnet's recursion formula. Obviously you will need to use a loop and work out how to increment the number of coefficients for each iteration. You can check your polynomials against the symbolic math toolbox function `legendreP`.

Part 2: Your Solution

Code (inc. Plots)

N.B.: Self-contained functions should be entered in the **Functions** section at the **bottom** of this livefile;

```
%-----M2.1-----v

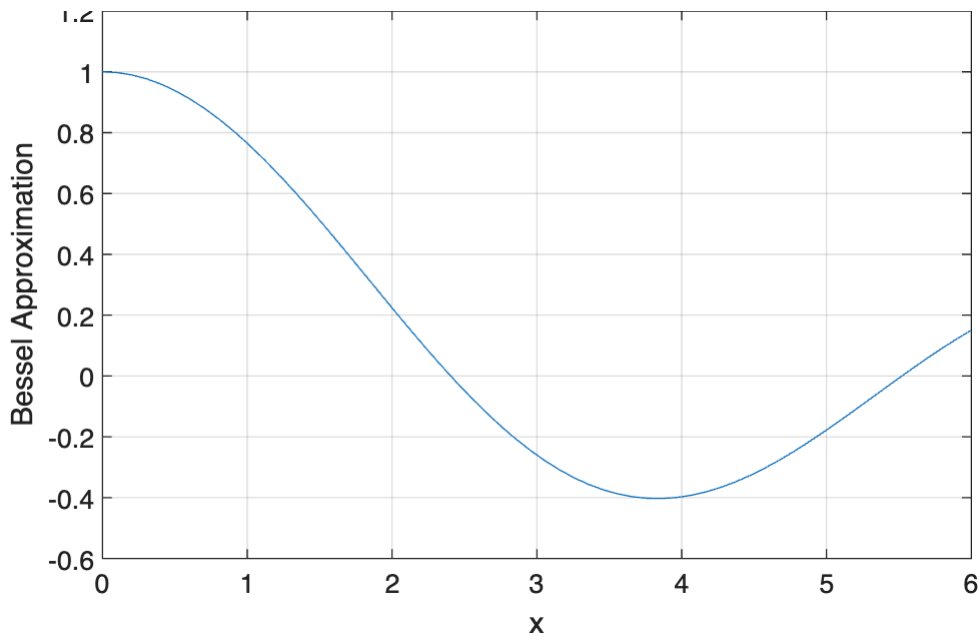
Y_trap = trapzJ(0,100,linspace(0,6,60));

%-----M2.2-----v

x_simp = linspace(0,6,60); %Used later
Y_simp = simpsonJ(0,1001,x_simp);

%-----M2.3-----v
n = 0; %Defining order of Bessel Function
N = 10; %Degree of Quadrature
x_legendre = linspace(0,6,60); %Defining x space
y_legendre = GaussJ(n,N,x_legendre); %Assignment of function output vector

figure
plot(x_legendre, y_legendre) %Plot of function vector against x vector
title('Gaussian Legendre Quadrature Approximation of Bessel Integral')
xlabel('x')
ylabel('Bessel Approximation')
grid on
```



```
%-----M2.4-----v
x_int = linspace(0,6,60); %used later
integral_vector = zeros(size(x_int));

fun = @(tau,x) cos(n*tau - x*sin(tau));
for i = 1:length(x_int)
    integral_vector(i) = (1/pi) * integral(@(tau) fun(tau,x_int(i)),0,pi);
end

%-----E2.1-----v

absE_trap = zeros(size(x_int)); %initializing so there is no constant size
change
absE_simp = zeros(size(x_int)); %initializing so there is no constant size
change
absE_legendre = zeros(size(x_int)); %initializing so there is no constant
size change

for n_nodes = [5,11,17,19,35,37,51] %Looping over each number of nodes we
wish to investigate
    Y_trap = trapzJ(0,n_nodes,linspace(0,6,60)); %Calling trapezium
function and initializing vector
    y_legendre = GaussJ(0,n_nodes,x_legendre); %Calling Gauss-Legendre
function and intializing vector
    Y_simp = simpsonJ(0,n_nodes,x_simp); %Calling simpson function and
intializing vector

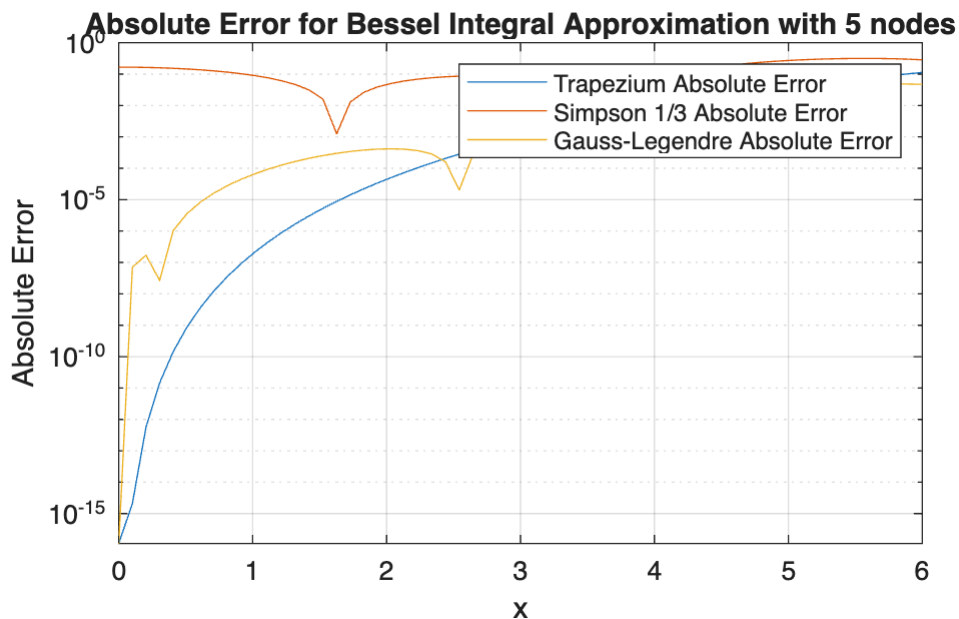
    absE_trap = abs(Y_trap - integral_vector); %Absolute error in the
trapezium vector and true vector
```

```

    absE_simp = abs(Y_simp - integral_vector); %Absolute error in the
    simpson vector and true vector
    absE_legendre = abs(y_legendre - integral_vector); %Absolute error in
    the gauss-legendre vector and true vector

figure
plot(x_int,absE_trap)
hold on
plot(x_int,absE_simp)
hold on
plot(x_int,absE_legendre)
hold off
set(gca,'YScale','log')
grid on
title("Absolute Error for Bessel Integral Approximation with "+
num2str(n_nodes) + " nodes")
legend('Trapezium Absolute Error','Simpson 1/3 Absolute Error', 'Gauss-
Legendre Absolute Error')
xlabel('x')
ylabel('Absolute Error')
end

```



```

%-----Code to Support Q2.1-----v
percision_error = 1;

fun = @(tau,x) cos(n*tau - x*sin(tau)); %Defining function to integrate

for i = 1:length(linspace(0,6,60))
    integral_vector(i) = (1/pi) * integral(@(tau) fun(tau,x_int(i)),0,pi);
end

```

```

%Trapezium
for node_num_trap = 1:100
    percision_error = max(abs(trapzJ(0,node_num_trap-1,linspace(0,6,60)) -
integral_vector));
    if percision_error < 10^-15
        disp("The number of nodes required to reach double percission is "
+ num2str(node_num_trap) + " for the trapezium method")
        break
    else
        continue
    end
end
end

```

The number of nodes required to reach double percission is 16 for the trapezium method

```

%Gauss
for node_num_gauss = 1:100
    percision_error = max(abs(GaussJ(0,node_num_gauss,linspace(0,6,60)) -
integral_vector));
    if percision_error < 10^-15
        disp("The number of nodes required to reach double percission is "
+ num2str(node_num_gauss) +" for the Gauss-Legendre Quadrature method")
        break
    else
        continue
    end
end
end

```

The number of nodes required to reach double percission is 17 for the Gauss-Legendre Quadrature method

```

%Simpson
for node_num_simp = 1:1000
    percision_error = max(abs(simpsonJ(0,node_num_simp,linspace(0,6,60)) -
integral_vector));
    if percision_error < 10^-15
        disp("The number of nodes required to reach double percission is "
+ num2str(node_num_simp) +" for the simpson method")
        break
    else
        continue
    end
end
end

```

Answers to Questions

Q2.1:

We can achieve double precision accuracy with the trapezium approximation of the Bessel integral using 16 nodes, implying a relatively low processing time. However, with the lack of output from the other two methods, we may conclude that there is no place at which double precision is reached. Having values of `node_num_gauss` of high value crashed the program, concluding that it was computationally not possible to reach machine precision. This is visible in the graphs as part of E2.1 - Gauss and Simpson behave asymptotically, and thus the set condition is never met.

Q2.2:

The trapezium rule error plots are missing data due to the logarithmic nature of the plot. At points where the absolute error is 0, MATLAB will attempt to plot $\log(0)$, which is undefined. This omits any regions on 0 error from the plot. This occurs with a greater number of nodes, as a greater number of nodes increases the accuracy with which the integral is evaluated, thus as the number of nodes increases, the number of areas omitted increases in turn.

Part 3: Bessel function roots [6 marks]

We now have two methods for evaluating Bessel functions at any x -value: interpolation of tables or numerical evaluation of integrals. Both of these methods will return a Bessel function at a series of input x -values. The advantage of the integral method is that it is not limited by the range of the data tables, neither in terms of x nor in terms of the order n . So it makes more sense to use this method. We can now proceed to determine the zeros and therefore find the resonant frequencies of the diaphragm.

Investigate the different methods of root finding covered in the course to find all of the roots on the interval for the **zeroth order Bessel function of the first kind**, $J_0(x)$, as follows:

- **M3.1:** Write a self-contained MATLAB function **called bisectionJ** that uses the bisection method. [1]
- **M3.2:** Write a self-contained MATLAB function **called newtonJ** that uses Newton's method. [1]
- **M3.3:** Write a self-contained MATLAB function **called secantJ** that uses the Secant method. [1]
- **M3.4:** Write a self-contained MATLAB function **called fzeroJ** that uses the built-in MATLAB function `fzero`. [1]

All functions should:

- take as input x and $J_n(x)$ computed using **M2.1** across the interval $0 \leq x \leq 6$.
- output all of the roots in the data to a tolerance of **1e-10** (in other words stop iterating when the solution is changing by less than the tolerance), if this is possible within a computation time of say 10 minutes (use Ctrl+C or Command+. to stop the computation if not).
- NOT use predetermined search intervals, but rather determine these from the function itself (so that in principle your code would function for any integer order/kind of Bessel function and/or outside the interval).
- all use the same method for determining the search interval.

You should evaluate the performance of your self-contained functions as follows:

- **E3.1:** Plot the approximated roots alongside the input function, using a different plot for each method, on the interval $0 \leq x \leq 6$. **[0.5]**
- **E3.2:** Compute and output the time required to compute all roots on the interval using e.g. `fprintf` to clearly state which values relate to which method. Also compute the average relative error for the two roots compared to the values in table 9.5 on page 409 of Arawmowitz and Stegun. **[0.5]**

Questions

- **Q3.1:** How do the various methods compare in terms of accuracy and computation time? Which method is best? **[0.5]**
- **Q3.2:** Using your preferred root finder with the preferred Bessel function generator from **M2.1** compute the frequencies (in Hz) of the 1st four modes of a circular diaphragm with the following properties: diameter, $D = 100\text{mm}$, thickness $h = 0.1\text{mm}$, density $\rho = 1000\text{ kg/m}^3$, membrane resultant $N = T/a = 12\text{ N/m}$. How do these values compare with those shown in the following video of a [Vibrating circular membrane](#) with approximately the properties given? **[0.5]**

Hints

One way to find the search interval is to start from the origin and find the first data point where the sign has changed, then search between that point and the next. You can then continue this to the next root.

Method 3.2: You will need to work out the derivative. This can be found from the series definition of the function, and you can also look it up at [DLMF: §10.6 Recurrence Relations and Derivatives • Bessel and Hankel Functions • Chapter 10 Bessel Functions \(nist.gov\)](#). You can also use the integral definition in section 2 to get the derivative. In all cases you will need to interpolate the derivative function at the same x-coords as the input Bessel function, and set this as an input to your root finder (so that you have the input function and it's derivative at every input point).

Part 3: Your Solution

Code (inc. Plots)

N.B.: Self-contained functions should be entered in the **Functions** section at the **bottom** of this livefile.

```
%-----M3.1 -----v
x_bi = linspace(0,6,60);
Bessely_vector = trapzJ(0,100,linspace(0,6,60));

zeros_bi = BisectionJ(x_bi,Bessely_vector)
```

```
zeros_bi = 1x2
    2.4048    5.5201
```

```
%-----M3.2 -----v
x_newton = linspace(0,6,60);
Bessely_vector = trapzJ(0,100,linspace(0,6,60));
zeros_newton = newtonJ(x_newton,Bessely_vector)
```

```
zeros_newton = 1x2
    2.4048    5.5201
```

```
%-----M3.3-----v
x_secant = linspace(0,6,60);
Bessely_vector = trapzJ(0,100,linspace(0,6,60));

zeros_sec = secantJ(x_secant,Bessely_vector)
```

```
zeros_sec = 2x1
    2.4048
    5.5201
```

```
%-----M3.4-----v

x_values_4fzero = linspace(0,6,60);
Y_trap = trapzJ(0,100,linspace(0,6,60));

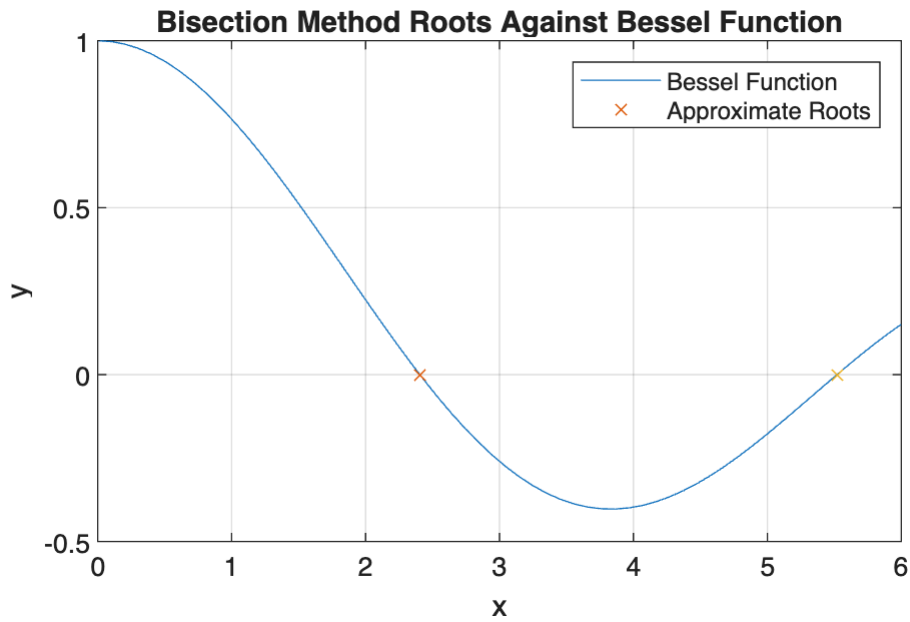
Bessely_vector = Y_trap;
x_fzeroJ = linspace(0,6,60);

roots_M31 = fzeroJ(0,x_fzeroJ,Bessely_vector)
```

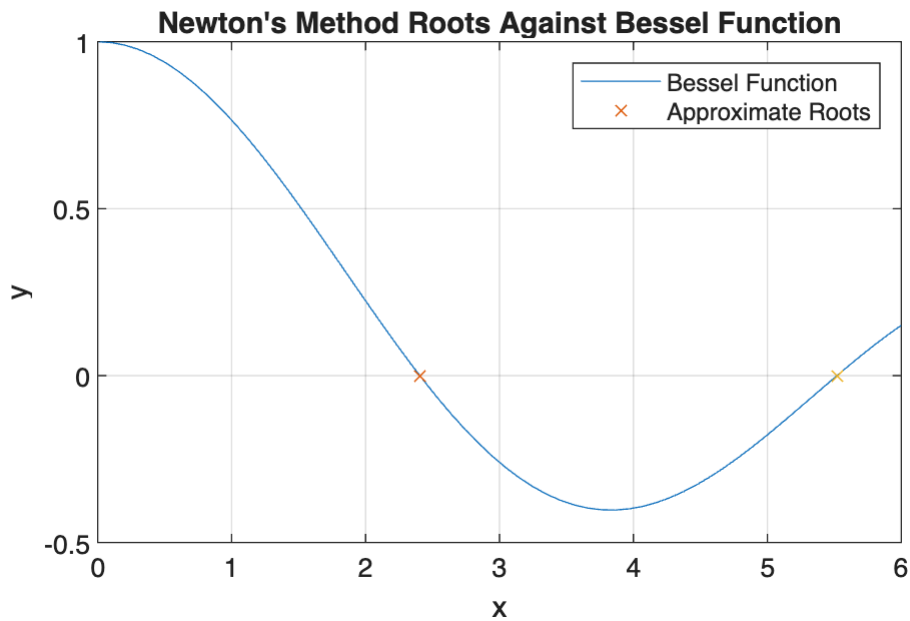
```
roots_M31 = 2x1
    2.4048
    5.5201
```

```
%-----E3.1-----v

%Bisection
figure
plot(x_bi,Bessely_vector)
hold on
scatter(zeros_bi,0,'x')
hold off
title("Bisection Method Roots Against Bessel Function")
grid on
xlabel('x')
ylabel('y')
legend("Bessel Function","Approximate Roots")
```

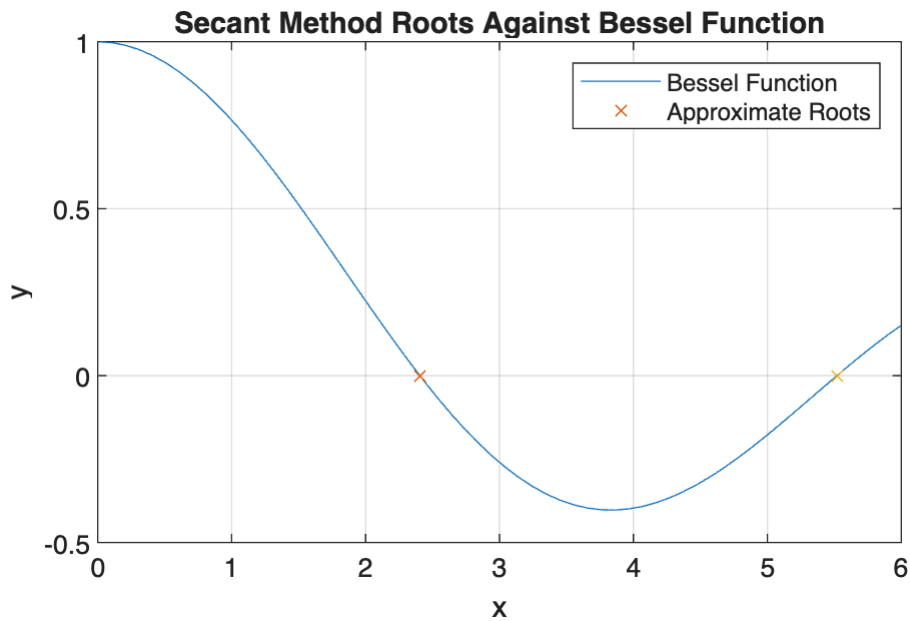
```
%newton
figure
plot(x_newton,BesselY_vector)
hold on
scatter(zeros_newton,0,'x')
hold off
title("Newton's Method Roots Against Bessel Function")
grid on
xlabel('x')
ylabel('y')
legend("Bessel Function","Approximate Roots")
```



```

%Secant plot
figure
plot(x_secant,Bessely_vector)
hold on
scatter(zeros_sec,0,'x')
hold off
title("Secant Method Roots Against Bessel Function")
grid on
xlabel('x')
ylabel('y')
legend("Bessel Function","Approximate Roots")

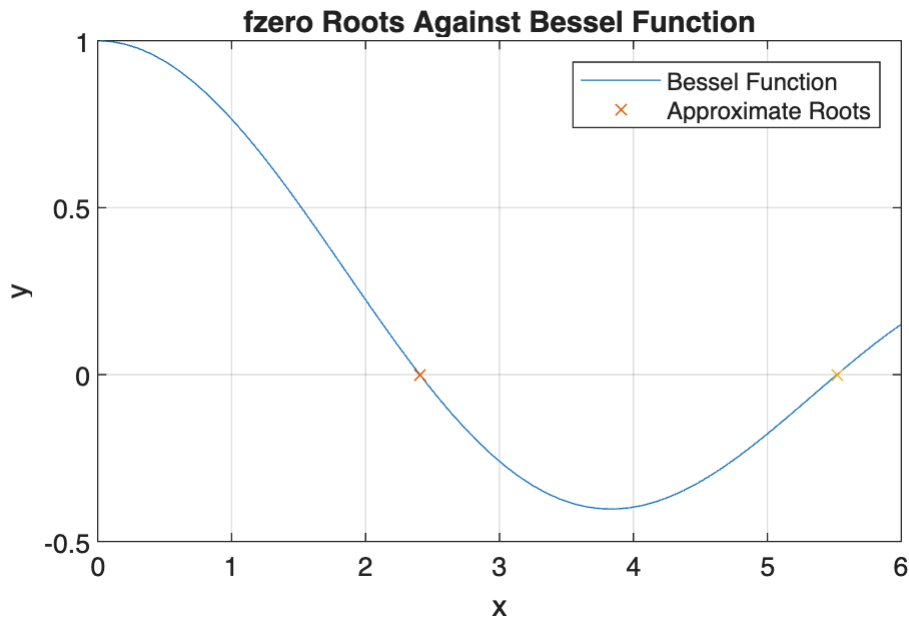
```



```

%fzero
figure
plot(x_values_4fzero,Bessely_vector)
hold on
scatter(roots_M31,0,'x')
hold off
title("fzero Roots Against Bessel Function")
grid on
xlabel('x')
ylabel('y')
legend("Bessel Function","Approximate Roots")

```



```
%-----E3.2-----v

%Computing times
tic %Starts stopclock timer
compute_roots_bi = BisectionJ(x_bi,Bessely_vector); %calls bisection
function
time_bi = toc; %reads elapsed time

tic %Starts stopclock timer
compute_roots_new = newtonJ(x_newton,Bessely_vector); %calls newton-raphson
function
time_new = toc; %reads elapsed time

tic %Starts stopclock timer
compute_roots_sec = secantJ(x_secant,Bessely_vector); %calls secant function
time_sec = toc; %reads elapsed time

tic %Starts stopclock timer
compute_roots_fzero = fzeroJ(0,x_fzeroJ,Bessely_vector); %calls fzeroJ
function
time_fzero = toc; %reads elapsed time

total_time = time_bi+time_fzero+time_sec+time_new; %sums times from each
method

%Printing Results
fileID = fopen('Root_Times.txt','w'); %Creating a txt file which will hold
results
fprintf(fileID,'Bisection Method Time: %f seconds\n', time_bi);
%Writing bisection time
```

```

    fprintf(fileID, 'Newton-Raphson Method Time: %f seconds\n', time_new);
%Writing Newton-Raphson time
    fprintf(fileID, 'Secant Method Time: %f seconds\n', time_sec); %Writing
Secant time
    fprintf(fileID, 'fZero Method Time: %f seconds\n', time_fzero); %Writing
fzero time
    fprintf(fileID, 'Total Elapsed Time: %f seconds\n', total_time);
%Writing total time
fclose(fileID); %closing file after it's been written on

open('Root_Times.txt') %opens file for reading

%Computing Average relative error

%The two roots in the region we are investigating
J0_true_root1 = 2.4048255577; %Data found from provided pdf
J0_true_root2 = 5.5200781103; %Data found from provided pdf

AError_new = mean([(abs(zeros_newton(1,1)-J0_true_root1))/
J0_true_root1,abs((zeros_newton(1,2)-J0_true_root2))/J0_true_root2]);
%Error in newton method
AError_bi = mean([(abs(zeros_bi(1,1)-J0_true_root1))/J0_true_root1,
(abs(zeros_bi(1,2)-J0_true_root2))/J0_true_root2]); %Error in bisection
method
AError_sec = mean([(abs(zeros_sec(1,1)-J0_true_root1))/J0_true_root1,
(abs(zeros_sec(2,1)-J0_true_root2))/J0_true_root2]); %Error in secant method
AError_fzero = mean([(abs(roots_M31(1,1)-J0_true_root1))/J0_true_root1,
(abs(roots_M31(2,1)-J0_true_root2))/J0_true_root2]); %Error in MATLAB fzero
method

disp("Newton average error " + num2str(AError_new)) %displaying result

```

Newton average error 2.1189e-12

```
disp("Bisection average error " + num2str(AError_bi)) %displaying result
```

Bisection average error 1.2822e-11

```
disp("Secant average error " + num2str(AError_sec)) %displaying result
```

Secant average error 2.1189e-12

```
disp("fzero average error " + num2str(AError_fzero)) %displaying result
```

fzero average error 2.1188e-12

```
%-----Code to support Q3.2-----v
```

```

density = 1000; %initializing density
radius = 0.05; %initializing radius
thickness = 0.1e-3; %initializing thickness
mem_res = 12; %initializing membrane resultant

```

```

for i = 0:2 %Investigating orders 0 through 2

    x_fzeroJ = linspace(0,20,500); %creating small increments to find
changes in sign
    BesselY_vector_i = trapzJ(i,100,x_fzeroJ); %Establishing our bessel co-
ordinate vector as trapezium approximation of order i

    roots_order_i = fzeroJ(i,x_fzeroJ,BesselY_vector_i); %Using fzero root
finding

    roots_order_i = roots_order_i(roots_order_i > 10^-6); %Removing any
possible root picked up at 0

    roots_order_i = roots_order_i(1:4); %Only counting first 4 roots

    omega_i = (roots_order_i/radius) * sqrt(mem_res/(density*thickness));
%Finding angular frequency

    freq_i = omega_i/(2*pi); %finding frequency

    fprintf('The frequencies for Bessel order %d are:\n', i); %printing
results

    disp(freq_i) %printing results
end

```

The frequencies for Bessel order 0 are:

83.8541
192.4802
301.7477
411.1602

The frequencies for Bessel order 1 are:

133.6081
244.6272
354.7397
464.5852

The frequencies for Bessel order 2 are:

179.0745
293.5017
405.1734
515.9215

Answers to Questions

Q3.1:

Regarding computation time and error, we get the following in ascending order for both fatcors:

Accuracy (From most accurate to least): [fzero, Secant, Bisection, Newton] - Low accuracy with Newton's method is due to use of trapzJ function to approximate bessel function for evaluation in both the function evaluation and first derivative evaluation. The Newton-Raphson is sensitive to small errors in these terms, thus leading to significant error differences.

Computation Speed (From fastest to slowest): [Secant, fzero, Newton, Bisection] - It should be noted that times generally change each iteration of the function, however this order is generally the result of averaging the times from a number of runs. Secant and fzero tend to be the two faster methods, whereas Newton and Bisection take longer.

Q3.2:

In the provided video, we see the following arrangements of the diaphragm: Order 0 mode 1, Order 1 mode 1, Order 2 mode 1, and Order 0 mode 2. The video shows the following respective frequencies: 82.2, 158, 217, and 227 - all in Hz. From the data collected by the code, we can achieve results of 83.9, 133.6, 179.0, and 192.5.

The results are somewhat similar, with absolute differences being in the range of around 30 Hz as a rough upper bound. Our approximation of (0,1) is very accurate, with an error of less than 1Hz, however for other modes this is not the case.

Your Self-Contained Functions

Part 1:

Intpoly

```
function y_for_Matrix = Intpoly(x_matrix,bessel_matrix_data) %creates
Intpoly function

X = zeros(length(x_matrix), length(x_matrix)); %Initializing zero matrix to
avoid size change

%Forming matrix of x to the power p up to the largest possible power. (See
Class 4 slide 13)
for i = 1:length(x_matrix) %Looping over x values
    for p = 1:length(x_matrix) %Incrementally increasing power
        X(i,p) = x_matrix(i)^(p-1); %Positioning entries
    end
end
A = X \ bessel_matrix_data(:); %A represents the vector of co-efficients.
This line multiplies the inverse of X by J0 (See Class 4 slide 14)

x_for_Matrix = linspace(0,5,101); %Creates a space at which the function
output can be evaluated at

A = flipud(A); %Flipping A, as polyval expects decending order in the
exponent of x
```

```

y_for_Matrix = polyval(A,x_for_Matrix); %Setting function output vector

end

```

Lagrangepoly

```

function L = Lagrangepoly(x_L_input,j0) %creating Lagrangepoly function
x_for_Lagrange = linspace(0,5,101); %Setting up 101 points

L = zeros(1,length(x_for_Lagrange)); %Initializing zero matrix to avoid
size change
LagrangeCoeff = zeros(1,length(x_for_Lagrange)); %Initializing zero matrix
to avoid size change

for j = 1:length(x_L_input) %Looping over all x values to be evaluated

    LagrangeCoeff = ones(1,length(x_for_Lagrange)); %Setting up
diagonalized coefficient matrix

    for k = 1:length(x_L_input) % Forming the Lagrange co-efficient
        if j ~= k

            LagrangeCoeff = LagrangeCoeff .* ((x_for_Lagrange - x_L_input(k))/
(x_L_input(j)-x_L_input(k))); %Implementing legrange polynomial
construction (See class 4 slide 35)
        end

    end

    L = L + j0(j) * LagrangeCoeff; %Summation to form function output (See
class 4 slide 35)

end

end

```

IntSpline

```

function spline_vector = IntSpline(x_spline_input,bessel_data) %creating
IntSpline function

%forming the matrix of fractions
main_diagonal = (2/3)*ones(49,1); %Forming main diagonal in coefficeint
matrix
shifted_diagonal = (1/6)*ones(48,1); %forming left and right diagonals

```

```

x_original = 0:0.1:5; %Data points otheriwse represented as x in this file
(bessel data x values)

fraction_matrix = diag(main_diagonal) + diag(shifted_diagonal,1) +
diag(shifted_diagonal,-1); %Forming the coefficeint matrix

b_matrix = zeros(49,1); %Initializing as zeros to prevent constant size
change

for i = 1:49 %Looping up to 49 (51 data points provided)
    b_matrix(i) = (bessel_data(i+2) - 2*bessel_data(i+1) + bessel_data(i))/
(0.1^2); %Forming matrix of linear equation results
end

second_der_vector = fraction_matrix \ b_matrix; %Finding matrix of unknowns
(See class 4 slide 64)
second_der_vector = [0 ; second_der_vector ; 0]; %Forming end constraints
of 2nd derivative = 0

spline_vector = zeros(size(x_spline_input)); %initializing to prevent
constant size change
for k = 1:length(x_spline_input) %Looping over all x values

    x_eval = x_spline_input(k); %These are the points at which x will be
evaluated (the 101 points required)

    j = find(x_original <= x_eval,1,'last'); %Finding where the evaluated
point maps best to the bessel x data
    if j == length(bessel_data) %Accouting to the case where j is equal to
vector length
        j = length(bessel_data)-1; %reducing j by 1
    end

    %Terms broken up for readability (See class 4 slide...
    term1 = second_der_vector(j)/6 * ((x_original(j+1)-
x_eval)^3) / (x_original(j+1)-x_original(j)) - ((x_original(j+1)-
x_original(j))*((x_original(j+1)-x_eval))) * second_der_vector(j)/6 ;
%first term of spline equation
    term2 = (second_der_vector(j+1)/6) * ((-x_original(j)+x_eval)^3) /
(x_original(j+1)-x_original(j)) - (x_original(j+1)-x_original(j))*((-
x_original(j)+x_eval))* second_der_vector(j)/6; %second term of spline
equation
    term3 = bessel_data(j)*(x_original(j+1)-x_eval)/(x_original(j+1)-
x_original(j)); %third term of spline equation
    term4 = bessel_data(j+1)*((x_eval-x_original(j))/(x_original(j+1)-
x_original(j))); %fourth term of spline equation

```



```

    spline_vector(k) = term1 + term2 + term3 + term4; %Spline full equation
    in the sum of the 4 terms (See class 4 slide 60)
end
end

```

Part 2:

trapzJ

```

function trapApprox = trapzJ(n,N,x_trap_values) %creating trapzJ function

tau = linspace(0,pi,N); %creating tau values for Bessel Integral evaluation
trapApprox = zeros(size(x_trap_values)); %initializing as 0 vector to
prevent constant size changes
for i = 1:length(x_trap_values) %looping over all desired x values
    f_tau = cos(n*tau - x_trap_values(i)*sin(tau)); %function within besse
integral
    trapApprox(i) = (1/pi) * trapz(tau, f_tau); %carrying out approximation
of integral using MATLAB trapz function with tau space and established
function
end

end

```

simpsonJ

```

function simpson_vector = simpsonJ(n,N,x_simpson_values) %creating function
simpsonJ
tau = linspace(0,pi,N); %creating tau space for Bessel Integral evaluation
h=pi/(N-1); %Initializing h
simpson_vector = zeros(size(x_simpson_values)); %initializing as 0 vector
to prevent constant size changes
for j = (1:length(x_simpson_values)) %looping over all desired x values
    f_tau = cos(n*tau - x_simpson_values(j)*sin(tau)); %function within
bessel integral

    sum1 = sum(f_tau(2:2:N-1)); %even sum

    sum2 = sum(f_tau(2:2:N-2)); %odd sum

    simpson_vector(j) = (1/pi) * ((h/3) * (f_tau(1) + 2*sum1 + 4*sum2 +
f_tau(end))); %Making use of Simpson's integral approximation (See class 3
slide 28)

end
end

```

LegendrePoly

gaussJ

```
function legendre_vector = GaussJ(n,N,x_legendre) %Defining the function

poly_coff_4_conv = [1 0 -1]; %coefficients of the polynomial x^2 -1

legendre_vector = zeros(size(x_legendre)); %initializing our final vector

for k = 1:length(x_legendre) %Loop over every x co-ordinate
    conv_result = 1; %resetting conv_result
    sum_wxf = 0; %resetting the sum

    for conv_iteration = 1:N %Loop over all N for our conv coefficients
        conv_result = conv(conv_result,poly_coff_4_conv); %Carrying out
        (x^2 -1)^N
    end
    L_N = conv_result; %resetting L_N
    for der_iteration = 1:N %Looping over N to differentiate our result N
        times
        L_N = polyder(L_N); %Carrying out differentiation
    end
    L_N = L_N/(2^N * factorial(N)); %Establishing our Legendre Polynomial

    eta = roots(L_N); %Finding roots of legendre polynomial
    w = 2./(1-eta.^2) .* (polyval(polyder(L_N),eta)).^-2; %Finding Weights

    for l_i = 1:N %Looping over all N to find sum of the product of weights
        and function output at root
        lambda = (pi/2) * (eta(l_i)+1); %Accounting for different limits
        f_eta = cos(n*lambda - x_legendre(k)*sin(lambda)); %Evaluating
        function at root
        sum_wxf = sum_wxf + w(l_i)*f_eta; %Summing the products of weights
        and functions
    end
    legendre_vector(k) = (1/pi)*(pi/2)*sum_wxf; %Appending the result to
    our output vector

end

end
```

Part3:

fzeroJ

```
%x_values_4fzero = linspace(0,6,60); %creating x values for fzero function
%Y_trap = trapzJ(0,100,x_values_4fzero); %creating y values that act as
evaluated points from Bessel function

function roots = fzeroJ(order,x_fzeroJ,Bessely_vector) %Creating fzeroJ
function
root_interval = []; %initializing as an array

%Finding the search regions
for i = 1:length(x_fzeroJ)-1 %looping over all x values
    a = x_fzeroJ(i); %establishing value of a
    b = x_fzeroJ(i+1); %value of b is set to be 1 greater than a

    if Bessely_vector(i)*Bessely_vector(i+1) < 0 %checking for a sign change
        root_interval = [root_interval;a b]; %Appends that interval to an
array
    else %Accounting for all other cases (no sign change)
        continue %keeps moving if nothing happens
    end
end

fzero_function_input = @(x) trapzJ(order,100,x); %creating function in x as
input for fzero
roots = zeros(size(root_interval,1),1); %initializing for no size change
%Finding roots
for j = 1:size(root_interval,1) %Looping over all entries in root_interval

    roots(j) = fzero(fzero_function_input,[root_interval(j)]); %function
output using fzero function

end

end
```

bisectionJ

```
function roots = BisectionJ(x_bi,Bessely_vector) %creating BisectionJ
function

%Search Region
root_interval = []; %initializing root interval
for i = 1:length(x_bi)-1 %looping over all x values
    a = x_bi(i); %establishing value of a
    b = x_bi(i+1); %value of b is set to be 1 greater than a

    if Bessely_vector(i)*Bessely_vector(i+1) < 0 %checking for a sign change
        root_interval = [root_interval;a b]; %Appends that interval to an
array
```

```

        else %Accounting for all other cases (no sign change)
            continue %keeps moving if nothing happens
        end
    end
end

roots = zeros(size(root_interval(1,:))); %initializing to prevent constant
size change
for j = 1:length(root_interval(1,:)) %looping over over all root interval
entries
    a = root_interval(j,1); %Setting a as jth row 1st cloumn
    b = root_interval(j,2); %Setting b as jth row 2nd cloumn
    while abs(b-a) > 10^-10 %Condition set for accuracy
        c=(a+b)/2; %c set as midpoint of and b
        if trapzJ(0,100,c)*trapzJ(0,100,a)<0 %Evaluating for sign change
            b=c; %update b in case of sign change
        else %No sign change
            a=c; %update a
        end
    end
    roots(j) = c; %function output
end
end
end

```

newtonJ

```

function roots = newtonJ(x_newton,Bessely_vector) %Create function newtonJ

%Search Interval
root_interval = []; %initializing root interval
for i = 1:length(x_newton)-1 %looping over all x values
    a = x_newton(i); %establishing value of a
    b = x_newton(i+1); %value of b is set to be 1 greater than a

    if Bessely_vector(i)*Bessely_vector(i+1) < 0 %checking for a sign change
        root_interval = [root_interval;a b]; %Appends that interval to an
array
    else %Accounting for all other cases (no sign change)
        continue %keeps moving if nothing happens
    end
end

%Finding roots
for j = 1:size(root_interval,1) %looping over all roots
    a = root_interval(j,1); %Setting a as jth row 1st column
    b = root_interval(j,2); %Setting b as jth row 2nd column
    c = (a+b)/2; %Setting c as midpoint of a and b (initial guess)

```

```

while true %Setting a condition controlled loop for accuracy

    d = c - trapzJ(0,1000,c)/dJ(0,1000,c); %Carrying out newton's
method (See Class 2 slide 22)
    if abs(d-c)< 1e-10 %Loop condition
        break %Exiting loop when condition met
    else %If condition to exit is not met
        c=d; %update guess
    end

end

roots(j) = d; %function output
end

end

function d_trapApprox = dJ(n,N,x_trap_values) %Forming derivative function
    tau = linspace(0,pi,N); %Setting tau values for Bessel funtion
    d_trapApprox = zeros(size(x_trap_values)); %initializing to prevent
constant size change
    for i = 1:length(x_trap_values) %Looping over all x values to evaluate
at each x
        df_tau = sin(n*tau - x_trap_values(i)*sin(tau)).* sin(tau);
%Derivative of the interior of the bessel integral
        d_trapApprox(i) = (1/pi) * trapz(tau, df_tau); %Approximating
integral using predefined trapezium function to output vector
    end

end

end

```

secantJ

```

function roots = secantJ(x_secant,BesselY_vector) %creating secantJ function

root_interval = []; %initializing as an array
for i = 1:length(x_secant)-1 %Looping from 1 to 1 less than number of
points
    a = x_secant(i); %establishing value of a
    b = x_secant(i+1); %value of b is set to be 1 greater than a

    if BesselY_vector(i)*BesselY_vector(i+1) < 0 %checking for a sign change
        root_interval = [root_interval;a b]; %Appends that interval to an
array
    else %Accounting for all other cases
        continue %keeps moving if nothing happens
    end
end

end

```

```

roots = zeros(size(root_interval,1),1); %Initializing to prevent constant
size changes

for j = 1:size(root_interval,1) %looping over all roots found
    a = root_interval(j,1); %Setting a as jth row 1st column
    b = root_interval(j,2); %Setting b as jth row 2nd column
    while abs(b-a)>10^-10 %Condition for accuracy
        c = b - trapzJ(0,100,b) * ((b-a)/(trapzJ(0,100,b)-
trapzJ(0,100,a))); %implementing secant method (See class 2 slide 29)
        a=b; %updating a
        b=c; %updating c
    end
    roots(j) = c; %Function output
end

end

```